

Sithu Soe - A17342422

Phyo Thant - A18498144

Title: DevLens → Acuity

One sentence description: An interactive, live technical interview website featuring an embedded AI coding assistant for candidates alongside a hidden real-time prompting-telemetry and evaluation dashboard for interviewers (Ship it with auth + live URL).

Past project reference: This is a completely new project. Repository: <https://github.com/PhyoTh/Acuity>. Live at <https://acuity-phyo.duckdns.org>.

This is the marked-up copy, updated for the final submission. Each item is tagged as implemented as written, changed, or dropped, with a pointer to where it lives in the code.

Planned technologies:

- Frontend Framework: Next.js, TailwindCSS, Monaco Editor **(Implemented as written)**
- Backend Server: FastAPI (Python), native Python asyncio WebSockets **(Implemented as written)** — `backend/app/routers/ws.py`
- State & Orchestration: LangGraph / LangChain **(Implemented as written)** — `backend/app/services/agent.py`
- Database & Cache: PostgreSQL (via Supabase Postgres), Redis **(Implemented as written)**
- Authentication: Supabase Auth **(Implemented as written)** — `backend/app/security.py`
- Deployment: **(Changed)** We planned Render but deployed on a free Oracle Cloud Always-Free VM running Docker Compose with Caddy for HTTPS and the websocket upgrade. We moved off Render because its always-on tier is paid and its free tier sleeps when idle, which breaks a live websocket interview. See `docker-compose.prod.yml`, `deploy/Caddyfile`, `DEPLOY.md`.

First deliverable:

(Implemented as written) A candidate and an interviewer share a live session, the candidate uses the AI assistant, and when the interview ends the dashboard shows an LLM scorecard. See `frontend/app/interview/[sessionId]/page.tsx`,

frontend/app/dashboard/[sessionId]/page.tsx, backend/app/routers/ws.py,
backend/app/services/scorecard.py.

Rough architecture for first deliverable:

- Auth & Session Manager (Supabase & Postgres) **(Implemented as written)** —
app/security.py, routers/auth.py, routers/interview.py, db/models.py
- Browser IDE Component (Next.js & Monaco) **(Implemented as written)** —
app/interview/[sessionId]/page.tsx, components/Editor/
- Real-Time Sync Gateway (FastAPI & Redis) **(Implemented as written)** —
app/routers/ws.py, redis_client.py, frontend/lib/ws.ts
- Agentic Chat Engine (LangGraph/LangChain) **(Implemented as written)** —
app/services/agent.py, llm.py
- Hallucination Injector Module **(Implemented as written, and extended)**: the interviewer now also picks the kind of flaw (logic, wrong-API, edge-case, inefficiency, security), not just the probability. We also merged the corruption into the agent's single generation call, so a corrupted turn costs one model call instead of two. See
app/services/hallucinator.py, app/services/agent.py
- Telemetry Logger **(Implemented as written)** — app/services/telemetry.py
- Scorecard Generator **(Implemented as written)** — app/services/scorecard.py

After first deliverable goals:

- Run candidate code against hidden test cases **(Implemented, changed)**: we used the Wandbox API instead of Piston, because Piston's public API became whitelist-only. See
app/services/executor.py and POST /sessions/{id}/run.
- Replay timeline that recreates the candidate's keystrokes and AI interactions
(Implemented as written) — components/Dashboard/ReplayTimeline.tsx + GET
/sessions/{id}/events
- Background LLM push-back questions on the interviewer's screen **(Implemented as written)** — app/services/pushback.py
- Pre-session config UI for hallucination probability, token limits, and guardrails
(Implemented as written) — components/CreateSessionForm.tsx
- Query quota on the chat box **(Changed)**: we replaced the per-query quota with a session-wide token budget that counts actual input + output tokens, which we felt was a truer

limit. See the budget logic in `app/routers/ws.py`.

- Cheat-mitigation paste flag for large external pastes (**Implemented as written**) — paste handling in `app/routers/ws.py` and the Monaco paste listener.

Additions beyond the proposal (built in response to review and staff feedback):

- Demo mode and a live public URL, so reviewers do not need Supabase, Anthropic, or Docker to try it (`DEMO_MODE / NEXT_PUBLIC_DEMO_MODE, POST /auth/demo-login`).
- Role-bound invite links, so a second interviewer cannot enter through the candidate link and skip the waiting room (separate candidate and co-interviewer codes).
- Multi-file projects, an interactive shell, and a candidate waiting room with admit and kick.